

Chapter 8

Arrays

Frequently, problems involve the manipulation of large amounts of data – lists, tables, and so on. For many of these problems, the array provides an appropriate structure for storing the data. In this chapter we examine techniques for both creating and manipulating arrays in Java.

8.1 Tables and Arrays

The table shown below gives the average Canadian retail price for one litre of regular gasoline for the period from 1980 to 2000.

Year	1980	1985	1990	1995	2000
Price(\$)	0.27	0.51	0.59	0.56	0.72

If we wanted to keep track of these data in a computer program, we *could* use five variable identifiers: `price1980`, `price1985`, `price1990`, `price1995`, and `price2000`. However, this is a bit cumbersome and, if we wanted to extend our data to keep a record of the price every year or even every month, then this solution would become totally unwieldy.

To overcome this problem, Java offers a very useful data structure, the *array* — a collection of data items of the same type. An array to hold our gasoline price data would require five *elements*. Java stores the elements of an array in consecutive locations in memory. An array has a single identifier with the elements being distinguished by an *index*. In Java, the values of the *indices* (plural of index) always start at zero. If we were to store our five gasoline prices in an array called `price`, then the values of the indices would range from zero to four. We can picture the array as shown in the following diagram. The identifiers of the individual elements of the array are `price[0]`, `price[1]`, ..., `price[4]`.

	0	1	2	3	4
price	0.27	0.51	0.59	0.56	0.72

In using arrays, it is important to distinguish between the elements of the array and the indices. In the `price` array, the elements are the different prices of gasoline while the indices mark the locations in the array of the elements. The elements here are floating point values while the indices are integers. It is also important to distinguish between the identifiers of the elements and the values of the elements. For the `price` array, the identifiers are `price[0]` and so on while the values are 0.27 and so on.

Example 1

The temperatures of a solution over a six-minute interval are recorded in the following table:

Time (min)	0	1	2	3	4	5	6
Temperature (°C)	32.0	27.2	24.4	22.7	21.6	21.0	20.6

An array `temp` could be used to record the temperatures of a solution each minute as follows:

	0	1	2	3	4	5	6
<code>temp</code>	32.0	27.4	24.4	22.7	21.6	21.0	20.6

The identifier of the array is `temp`.

The identifiers of the elements are `temp[0]`, `temp[1]`, ..., `temp[6]`.

The values of the elements are 32.0, 27.4, ..., 20.6

The indices are 0, 1, ..., 6.

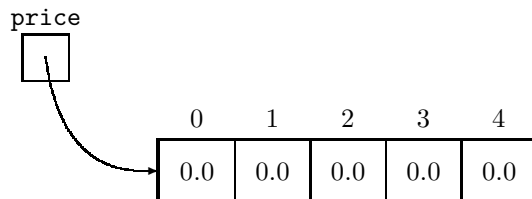
In Java, an array is a kind of object. To create an array, we proceed in a way very similar to that used in creating any object. For example, to create an array to hold the gasoline price data, we could begin with the declaration

```
double[] price;
```

This creates the variable `price` and determines that the type of `price` is a reference to an array of `double` values. As with other objects, the declaration, by itself, does not actually create an array. To get Java to allocate memory for an array of five double values, we can now write

```
price = new double[5];
```

Pictorially, the result of this statement is:



Notice the value 0.0 shown in each element of the array. Arrays are automatically initialized when we allocate space to them. If the elements

are numeric, they are initialized to zero (of the appropriate type); if they are **char** values, they are initialized to the *null character* with representation consisting of 16 zero bits; if they are **boolean** they are initialized to **false**; if they are references, they are initialized to **null**.

Again, as with any object, we can combine the declaration of the array reference variable and the allocation of memory into one statement. For the **price** array, we could have written

```
double[] price = new double[5];
```

to declare, allocate memory for, and initialize the **price** array in one statement.

This can be done in general. A statement of the form

```
<type>[] <identifier> = new <type>[<expression>];
```

declares that **<identifier>** is an array whose components are of type **<type>** and also allocates space in memory for such an array.¹ The size of the array is determined by the value of **<expression>**. This is often simply a positive integer constant but it can be any expression that has a positive value and is of any integer type other than **long**.

Example 2

The following fragment declares an **int** array **list** of size 20.

```
byte b = 10;
int[] list = new int[2*b];
```

Once an array has been declared, its size is fixed for the duration of its existence. Every array object has a **length** field whose value can be obtained by appending **.length** to the identifier of the array. The value of an expression of the form **<identifier>.length** is the number of elements in the array. Since arrays are always indexed from zero, the length of an array is one greater than the highest index in the array.

Java has an alternative form of array declaration that allows us to initialize an array with any values that we like. The next example illustrates this feature.

¹Java also allows us to write array declarations in the form **<type> <identifier> []** with the square brackets following the identifier but we will not be using that form.

Example 3

The following statement declares an `int` array called `primes` and initializes it with the values of the first ten prime numbers.

```
int[] primes = {2,3,5,7,11,13,17,19,23,29};
```

Notice in the example the absence of the keyword `new` and the use of brace brackets rather than square brackets on the right side of the assignment operator. In addition, the size of the array is not stated explicitly; Java knows that the array is of length ten because there are ten values inside the brace brackets. The values inside the brace brackets need not be constants; they can be expressions which will be evaluated at the time that the program is executed. The only restriction on these expressions is that they all be of the same type since (as we said at the beginning of this section) arrays are collections of values of the same type.

Although Java's arrays are always indexed from zero, sometimes the values that we are dealing with are more naturally indexed from some other value. Suppose, for example, that we wanted to store the values of a sequence that has terms $t_1, t_2, \dots, t_n$. We could use an array of length n but then t_i would be stored in the location with index $i-1$. An alternative (that we favour) is to declare the array to be of size $n+1$ and then store t_i at location i with the location having index zero being unused.

Example 4

Suppose that we wanted to store the sequence with terms

$$t_1 = \frac{1}{2}, t_2 = \frac{1}{4}, \dots, t_5 = \frac{1}{32}$$

in an array of `double` values. We could create and initialize such an array by writing

```
double[] terms = {0.0,0.5,0.25,0.125,0.0625,0.03125};
```

This would store t_i at the location with index i . The location with index zero would be unused.

Exercises 8.1

1. An array to store marks for twenty students has been declared as follows:

```
int[] marks = new int[20];
```

- What is the array identifier?
 - What is the identifier of the first element in the array?
 - What value is stored in each element by the declaration?
 - What is the value of `marks.length`?
 - What are the indices of the array?
2. How much space (in bits) would be required to store the elements of each array?
- `int[] a = new int[20];`
 - `double[] b = new double[100];`
 - `float[] c = new float[50];`
 - `boolean[] d = new boolean[1000];`
3. Write declarations to create arrays that would be appropriate for storing the indicated data.
- the numbers of votes cast for five candidates in an election
 - the answers to a twenty-question true/false quiz
 - average family size in the years 1900, 1910, ..., 2000
4. (a) Write a statement that creates and initializes an array `terms` of `double` values to store the terms of the sequence

$$t_1 = \frac{1}{2}, t_2 = \frac{2}{3}, \dots, t_6 = \frac{6}{7}$$

- What is the value of `terms.length`?
5. The table gives atomic masses of the eight lightest elements listed according to atomic number.

Atomic Number	1	2	3	4	5	6	7	8
Atomic Mass	1.0	4.0	6.9	9.0	10.8	12.0	14.0	16.0

Suppose that the data in this table were to be represented by the array `mass` declared by the statement

```
double[] mass = {0,1,4,6.9,9,10.8,12,14,16};
```

- (a) What is the value of `mass[2]`?
- (b) What is the value of `mass[5]`?
- (c) What are the possible values of the indices of the array?
- (d) What is the identifier of the element whose value is 6.9?
- (e) Of what type are the elements?
- (f) What is the value of `mass.length`?

8.2 Using Arrays

Having seen how to create arrays, let us now examine some of the ways that we can operate on them and their elements.

The elements of an array can be treated like any other variable of that type. We can assign values to them, print them, use them in expressions, and so on.

Often we want to perform some operation on every element of an array. We sometimes refer to this as *indexing* through the array.

Example 1

If we wanted to create an array called `flags` containing 100 boolean values all set to `true` (rather than the standard initialization to `false`), we could write

```
boolean[] flags = new boolean[100];
for (int i = 0; i < flags.length; i++)
    flags[i] = true;
```

In the example, we used a `for` statement to index through the array. We could have used a `while` or a `do` but the `for` is simplest. It is almost always the case that a `for` is the best way to index through an array. Notice

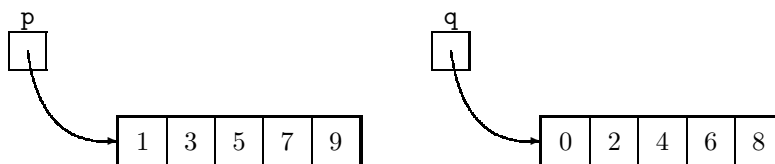
also the use of `flags.length` to determine the upper bound. This is more reliable and probably clearer to a reader than using the value 100.

Finally, note the way that we have written the condition that controls the loop — as `i < flags.length`. The value of `flags.length` is the size of the array — the number of elements in it. As we have already noted, since an array is indexed from zero, the index of the last element is one less than the value of its length. If we were to write the condition as `i <= flags.length`, then we would be trying to access an element off the end of the array. If we attempt to use an index value that is out of the range of an array, Java rewards us by throwing an `ArrayIndexOutOfBoundsException`.²

Because arrays, like other objects, are reference types, they behave in similar ways when we try to manipulate them. As an example, suppose that we make the following declarations:

```
int[] p = {1,3,5,7,9};
int[] q = {0,2,4,6,8};
```

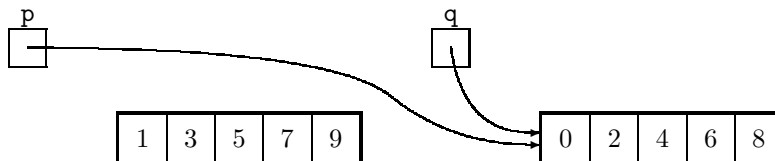
to give us the situation pictured in the next diagram.



If we were now to write the assignment statement

```
p = q;
```

this would copy the value from the cell labelled `q` in the diagram to the one labelled `p`. The effect of this would be to have `p` refer to the same array as `q`. Since there is no longer any reference to the array of odd numbers, they are no longer accessible to the program.



Comparison of arrays is like comparison of other objects in that, if `p` and `q` are array variables, the expression `p == q` will be true if and only if

²See Appendix E for notes on exception handling.

both `p` and `q` are referring to the same array cells. It is *not* a test of the equality of the elements of two distinct arrays. If we wanted to test for equality of the contents of two arrays, we could use a loop that indexed through the arrays comparing corresponding pairs.³

Arrays, like all objects, can be parameters of methods and can be returned by methods.

Example 2

The method `join` creates a new array that contains the elements of each of the method's parameters and returns a reference to that array.

```
public static double[] join (double[] a, double[] b)
{
    double[] result = new double [a.length + b.length];
    int i, j;
    for (i = 0; i < a.length; i++)
        result[i] = a[i];
    for (j = 0; j < b.length; i++, j++)
        result[i] = b[j];
    return result;
}
```

As usual, Java passes parameters by value but, with arrays, the values that are passed are references to the arrays. This means that methods have the ability to alter the contents of arrays that are passed as parameters.

Example 3

The method `swap` shown here will switch the values of two elements in an array of `double` values.

```
public static void swap (double[] list, int i, int j)
{
    double temp = list[i];
    list[i] = list[j];
```

³Java contains a method `Arrays.equals` in the `java.util` package that compares equality of content of arrays.

```

    list[j] = temp;
}

```

To show the effect of `swap`, suppose we have an array `masses` shown below.



If we then write the statement

```
swap(masses, 1, 4);
```

the values at `masses[1]` and `masses[4]` will be switched.



Exercises 8.2

1. What would be printed by the following program fragment?

```

int[] list = new int[4];
for (int i = 0; i < list.length; i++)
    list[i] = 3 - i;
System.out.println(list[1]+2);
System.out.println(list[1+2]);
System.out.println(list[1]+list[2]);

```

2. Suppose that an array `sample` has been declared as follows:

```
int[] sample = new int[SIZE];
```

Write one or more statements to perform each task.

- (a) Initialize all elements of the array to one.

- (b) Switch the values at either end of the array.
 - (c) Change any negative values to positive values (of the same magnitude).
 - (d) Set the variable `sampleSum` to the sum of the values of all the elements.
 - (e) Print the contents of the odd-numbered locations.
3. Write a method `max` that has one `double` array parameter. The method should return the value of the largest element in the array.
 4. Complete the definition of the method `equals` so that it returns true if and only if its two array parameters contain equal elements.

```
public static boolean equals (double[] a, double[] b)
```

5. Write a program that repeatedly prompts the user to supply scores (out of 10) on a test. The program should continue to ask the user for marks until a negative value is supplied. Any values greater than ten should be ignored. Once the program has read all the scores, it should produce a table with the following headings:

Score	# of Occurrences
-------	------------------

The program should then calculate the mean score, rounded to one decimal place.

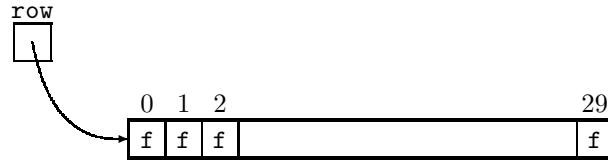
8.3 Multi-Dimensional Arrays

Suppose that, because of your growing reputation as an expert with computers, you have been asked to assist the local little theatre group in maintaining records of seats sold for their upcoming play. The auditorium that the group uses has twenty-five rows, each of which contain thirty seats. To keep track of sales, it would be useful to have an array of boolean values in which an element has the value `true` if and only if the seat has been sold.

Sales of seats in a single row could be monitored using a boolean array of size thirty. This could be declared with the statement

```
boolean[] row = new boolean[30];
```

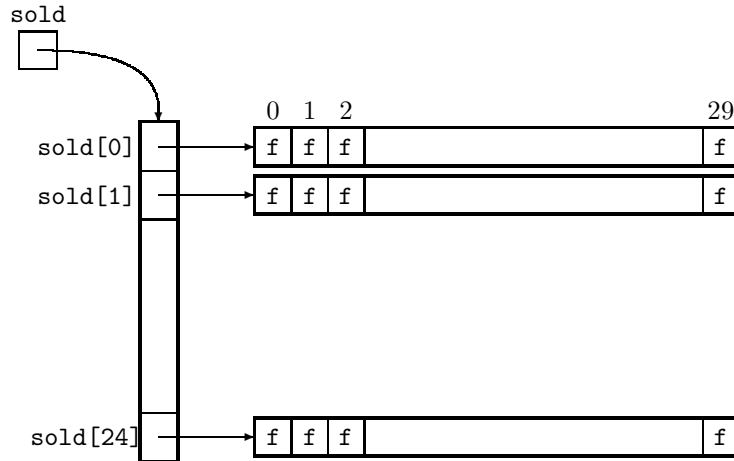
to produce the following result.



To monitor sales for the entire auditorium, we want one of these arrays for each row. We can do this easily by writing

```
boolean[] [] sold = new boolean[25][30];
```

The results of this declaration are shown in the next diagram.



Although this may be surprising at first, analysis may make it seem perfectly reasonable. Recall that the general form of a declaration of an array was

```
<type>[] <identifier> = new <type>[<expression>];
```

Comparing this form with our declaration of the `sold` array, the `<type>` of the components of `sold` is `boolean[]`; each element of the `sold` array is of type `boolean[]`, a reference to an array of boolean values. As the diagram indicates, the identifiers of these references are `sold[0]`, `sold[1]`, ..., `sold[24]`.

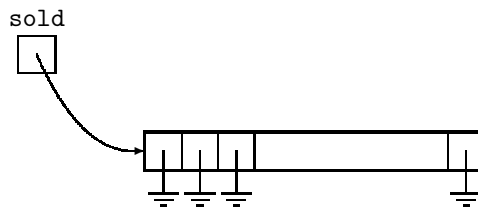
Each `sold[i]` value refers to a boolean array, each of whose elements are initialized to `false` (indicated by the letter `f` in the diagram). The elements of the boolean arrays are identified by two indices — the first for the index of the array reference and the second for the index of the element

within its array. For example, the element in the upper left hand corner of the diagram has identifier `sold[0][0]` while the element in the lower right hand corner has identifier `sold[24][29]`.

We could have created this structure in stages. As a first step, we could have written

```
boolean[] [] sold = new boolean[25] [];
```

to create `sold`, a reference to an array of 25 variables of type `boolean[]`. As usual, when we create a new array, the values are initialized. Since the type of the values is a reference type, each one is initialized to `null`.



Now, to create the arrays to represent each row, we can use a loop.

```
for (int i = 0; i < sold.length; i++)
    sold[i] = new boolean[30];
```

The loop would be executed 25 times (the value of `sold.length`). Each time around, one more array of 30 elements would be generated and initialized. The final result would be what we want: 25 arrays of 30 boolean elements representing the seats sold in the auditorium. An array of arrays like this one is known as a *two-dimensional array*.

Often, we want to use two-dimensional arrays to represent values in a simple rectangular table. In such cases, we can think of a two-dimensional array as consisting of rows and columns. If we do this, it is customary to think of the index representing a row preceding that representing a column.

Example 1

A table of `int` values with 3 rows and 6 columns can be created using the declaration

```
int[] [] table = new int[3][6];
```

As we have seen, this really creates three arrays each containing six elements but we can think of it as shown in the following diagram.

		columns					
		0	1	2	3	4	5
rows	0	0	0	0	0	0	0
	1	0	0	0	0	0	0
	2	0	0	0	0	0	0

Just as `for` statements are usually used to index through a one-dimensional array, nested `for` statements are usually used to index through a two-dimensional array. For the array `table` shown in Example 1 we can initialize the elements to one using the following fragment.

```
for (int row = 0; row < table.length; row++)
    for (int col = 0; col < table[0].length; col++)
        table[row][col] = 1;
```

Notice the way in which the upper bounds of the loops are written. The number of rows in `table` is given by `table.length` while the number of columns in row zero (and all other rows) is given by `table[0].length`.

Two-dimensional arrays can be initialized as they are being declared, in a manner similar to that used for one-dimensional arrays.

Example 2

Suppose we want to create a table with three rows and four columns, initialized as shown in the diagram.

5	2	7	3
6	8	7	9
4	5	2	3

We can do this with the following statement.

```
int[] [] scores = {{5,2,7,3},
                   {6,8,7,9},
                   {4,5,2,3}};
```

On the right side of the assignment statement, the elements of each row of the array are enclosed by brace brackets and separated by commas. Similarly, the sets of values for each row are themselves enclosed by brace brackets and separated by commas. The statement could have been written

on one line but we wrote it the way that we did to make the structure of the array clearer to a human reader.

The one-dimensional arrays that compose a two-dimensional array need not all be of the same length, as the next example illustrates.

Example 3

The following fragment creates a triangular array of `double` values with one element in the first row, two in the second, and so on for a total of five rows.

```
double[] [] triTable = new double[5] [];
for (int row = 0; row < triTable.length; row++)
    triTable[row] = new double[row+1];
```

We can picture the resulting array as follows:

		columns				
		0	1	2	3	4
rows	0	0				
	1	0	0			
	2	0	0	0		
	3	0	0	0	0	
	4	0	0	0	0	0

Array declarations like the one for `triTable` in Example 3, in which the second dimension's size was unspecified, are legal. However, it is *not* legal to have the first dimension's size unspecified. For example, the declaration

```
int[] [] badArray = new int[] [20];
```

is not permitted.

Non-rectangular two-dimensional arrays are sometimes called *ragged arrays*. We can initialize ragged arrays in their declarations, just as we did for rectangular arrays.

Example 4

The statement

```
int[] [] ragged = {{4,3,7},
                  {5,2},
                  {7,8,1,4}};
```

creates the array whose elements are illustrated in the following diagram.

4	3	7	
5	2		
7	8	1	4

To process the elements of a ragged array, we must change our loop structure slightly to take account of the fact that the length of each row may be different.

Example 5

The following fragment could be used to find the sum of the elements of a ragged two-dimensional array like the one shown in the previous example. Notice that the upper bound of the inner loop now depends on the length of each row of the array.

```
int sum = 0;
for (int row = 0; row < ragged.length; row++)
    for (int col = 0; col < ragged[row].length; col++)
        sum += ragged[row][col];
```

Multi-dimensional arrays are not limited to two dimensions. We can create three-dimensional arrays, four-dimensional arrays, or arrays of even higher dimensions. These higher-dimensional arrays are usually rectangular, but they need not be. We suggested that a two-dimensional rectangular array could be visualized as a table with rows and columns. Similarly, a rectangular three-dimensional array can be visualized as a set of tables,

perhaps a book containing a number of tables, one on each page. The indices could then be thought of as representing the pages, rows, and columns (in that order). A four-dimensional array could then be thought of as a number of volumes of books, all of the same size, with an index specifying a volume, page, row, and column. As with two-dimensional arrays, we need not specify all the dimensions in a declaration of a higher-dimensional array.

Example 6

The following declarations are valid.

- (a) `char[] [] [] goodOne = new char[5][6][3];`
This creates an array of $5 \times 6 \times 3 = 90$ `char` elements or, more accurately, five sets of six sets of three-element `char` arrays.
- (b) `byte[] [] [] goodTwo = new byte[20] [] [];`
This creates 20 arrays of type `byte[] []`. Each of these 20 arrays (called `goodTwo[0]`, `goodTwo[1]` and so on) will be initialized to `null`.
- (c) `double[] [] [] goodThree = new double[50][100] [];`
This creates an array of 50 references to arrays of 100 references to arrays with elements of type `double`. These 5000 array references will be set to `null`.

Example 7

These declarations are not valid.

- (a) `float[] [] badOne = new float[] [50];`
This attempt to create an unspecified number of arrays that are each of type `float[50]` will fail. The first dimension must always be specified.
- (b) `boolean[] [] [] badTwo = new boolean[10] [] [25];`
Although the first dimension has been specified here, the second and third dimension show the same pattern that was exhibited in `badOne`.

Finally, we note that, just as one-dimensional array references can be used as parameters and return values of methods, so too can multi-dimensional array references. Of course, the dimensions of arguments and parameters must match and their types must be appropriate. It is also possible to pass sub-arrays to methods, as the next example illustrates.

Example 8

Suppose that a program had a method with the following header:

```
public static void process (double[] row)
```

Suppose also that in our `main` method, say, we had made the following declaration:

```
double[] [] table = new double[40][60];
```

We could then call `process` from `main` by writing

```
process(table[20]);
```

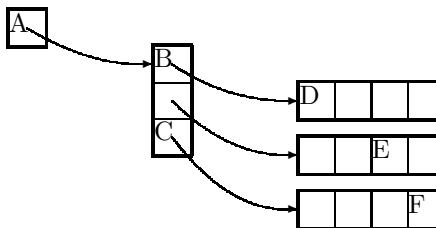
This works because `table[20]` is a reference to a one-dimensional array of `double` values and that exactly matches the type of the parameter `row`.

Exercises 8.3

1. The diagram shows an array declared by the statement

```
int[] [] a = new int[3][4];
```

State the identifier of each cell marked by a letter.



2. How many elements would there be in each of the arrays created by the following declarations?

(a) `double[] [] first = new double[25][40];`

- (b) `boolean[][][] second = new boolean[3][6][50]`
- (c) `char[][] third = new char[60][40];`
- (d) `long[][][] fourth = new long[5][10][20];`

3. Suppose that the following declarations have been made:

```
int a[][] = {{4,2,7},
             {3,9,1}};
int i, j;
```

Determine what would be printed by each fragment.

- (a)

```
for (i = 0; i < a.length; i++)
{
    for (j = 0; j < a[0].length; j++)
        System.out.print(a[i][j]);
    System.out.println();
}
```
- (b)

```
for (i = 0; i < a[0].length; i++)
{
    for (j = 0; j < a.length; j++)
        System.out.print(a[j][i]);
    System.out.println();
}
```
- (c)

```
for (i = a.length - 1; i >= 0; i--)
{
    for (j = 0; j < a[0].length; j++)
        System.out.print(a[i][j]);
    System.out.println();
}
```

4. For the array given in the previous question, write a fragment that would print the elements of the array in the form

```
17
92
34
```

5. Write a method `sum` having one `double[][]` parameter. The method should return the sum of the elements of the array passed to it. You may assume that the array is rectangular.

6. Write a method `max` that will return the maximum value of the elements in a two-dimensional array of `int` values. Do *not* assume that the array is rectangular.
7. Write a method `print` that could be used to print a two-dimensional ragged array of `int` values. Each row of elements should be printed on its own line with one blank between each element.
8. Write a method `size` that has one `int [][]` parameter. The method should return the number of elements in the array. Do not make any assumptions about regularity of the array.

8.4 Arrays of Objects

We said earlier that the elements of an array could be of any type. This includes the possibility that elements can be objects. To illustrate this, consider a class that could be used to represent complex numbers. We could begin to define such a class by writing

```
class Complex
{
    private double re;
    private double im;
}
```

To create an array of `Complex` objects, we could proceed as follows. In our `main` method, say, we could write

```
Complex[] points = new Complex[100];
```

This would create an array of 100 values (`points[0]`, `points[1]`, ..., `points[99]`) of type reference to `Complex`. As with the declaration of any array of references, each of these would be initialized to `null`. To actually create 100 `Complex` objects, we could use the following loop.

```
for (int i = 0; i < points.length; i++)
    points[i] = new Complex();
```

Assuming that the `Complex` class has a `toString` method, we could print the value of the fifth object in the array by writing

```
System.out.println(points[4]);
```

Objects can be very useful in situations that call for a number of arrays of the same size. As an example, suppose that we are collecting data on a group of children. We might have an array `name` to store their names, another array `age` to store their ages in years, a third array `height` to store their heights in metres, and a fourth array `sex` to store their sex as a character (M or F). These could be declared as follows:

```
String[] name = new String[STUDY_SIZE];
int[] age = new int[STUDY_SIZE];
double[] height = new double[STUDY_SIZE];
char[] sex = new char[STUDY_SIZE];
```

Arrays of the same length, carrying data on different aspects of the same problem, are sometimes called *parallel arrays*. Rather than using four parallel arrays for our study of child development, we could use a single array of objects.

Example 1

We could begin to define a class that could be used for a study of children's development as follows:

```
class Child
{
    private String name;    // family, given
    private int age;        // in years
    private double height;  // in metres
    private char sex;       // M or F
}
```

Now, to create a structure to store data on the children, we could write

```
Child[] patient = new Child[STUDY_SIZE];
```

The data for the first child in the study would now be called `patient[0]`.

The age of this child would be `patient[0].age`.

There are a number of advantages to using an array of objects rather than multiple arrays of data.

- A method that processes data about one object can be given a single object parameter rather than many item parameters.
- If we modify the program later to include different kinds of data, we need only change the fields of the object class; no parameters need to be added to our methods.
- By making the fields private, we can ensure (by using well-tested accessor and mutator methods) that the data will be handled appropriately.

Although the elements of an array must all be of the same type, if we are dealing with objects, we can use some of the ideas developed in Chapter 7 to get around this restriction and store objects of various types in the same array.

One way to do this is to declare an array to be of type `Object`. Since `Object` is the superclass of all object classes, an object of any type must have an `Object` part and can, therefore, be stored in an array of type `Object`. The actual type of an element at execution time can be determined by using the `instanceof` operator.

Another approach is to create an abstract superclass of all the classes whose objects we want to have in an array. This is done in the next example where objects representing various shapes are stored in the same array.

Example 2

Suppose we have objects of type `Circle` and type `Square` that we want to store in the same array. We could do so if we create an abstract class `Shape` whose partial definition might be

```
abstract class Shape
{
    public abstract double area ();
    ...
}
```

We could then begin to define the `Circle` and `Square` classes as follows.

```
class Circle extends Shape
{
    private double radius;
    public double area ()
```

```

    {
        return Math.PI*radius*radius;
    }
    ...
}

class Square extends Shape
{
    private double side;
    public double area ()
    {
        return side*side;
    }
    ...
}

```

Now we can create an array of elements whose type is **Shape** and store either **Circle** or **Square** objects in that array. Whether the object is a **Circle** or **Square** can, as before, be determined at execution time by using the **instanceof** operator.

Exercises 8.4

1. For the **points** array discussed in this section, write a fragment that would set all the values of the array to represent the complex number $1 + i$. You may assume the existence of a constructor with header `public Complex (double re, double im)`.
2. (a) For the child development study discussed in this section, write a method that could be added to the **Child** class that prints the data (name, age, height, and sex) for a child on two lines with the name on the first line and the other data on the second line. Print the height in centimetres, rounded to the nearest centimetre.
 - (b) Write a fragment that uses this method to print the data on all the children.
 - (c) Write a method that would produce the same output as the method shown in part (a) if we had chosen to use parallel arrays rather than an array of objects to represent our data.

3. Assuming that a `Fraction` class has been defined as shown, write a fragment that could be used to create an array of 100 `Fraction` objects representing the fractions

$$\frac{0}{1}, \frac{1}{2}, \frac{2}{3}, \dots, \frac{99}{100}$$

```
class Fraction
{
    private int num;
    private int den;

    public Fraction (int n, int d)
    {
        num = n;
        den = d;
    }
}
```

4. Suppose that a program has an array `shapes` whose elements are of type `Shape`, as described in Example 2. Write a fragment that will print, for each element in the array, a message identifying the type of shape (circle or square) represented by that object and its area.

8.5 Partially Filled Arrays

Up to now, we have assumed that arrays are always exactly the right size — the number of elements is equal to the length of the array. Often, however, we are faced with situations where the size of an array is not known in advance. To handle such situations, we must modify our approach slightly.

A simple solution to the problem is to create an array whose length is great enough to hold the maximum number of items that we will ever need and then have another variable keep track of the current size of the array — the number of cells that are currently occupied by data. Although it is not necessary to do so, we think that it is a good idea to always identify the current size in a consistent way. We will always identify these variables using the array name followed by `Size`.

Example 1

Assuming that `MAX_LIST_LENGTH` has a value of 100, then the fragment

```
double[] list = new double[MAX_LIST_LENGTH];
int listSize = 0;
```

creates a `double` array called `list` whose maximum size is 100 (available in the variable `list.length`) and whose current size is initialized to zero (available in the variable `listSize`).

To add a new element to such an array, we must update the current size when an item is added. In addition, before adding a new item, we must check that there is room in the array. Even though we set the array's length to be as large as we would *ever* need, it is almost certain that, if a program is run often enough, all estimates of maxima and minima will eventually be exceeded.

If an array is full when we try to add an element, we can solve that problem by making the array larger. We cannot do this directly during program execution since an array's size is fixed at declaration. We can, however, do it indirectly in the following way. We first create a new array that is larger than the original, then copy all the elements of the original array into the temporary one, and finally set the original array reference to the new array.

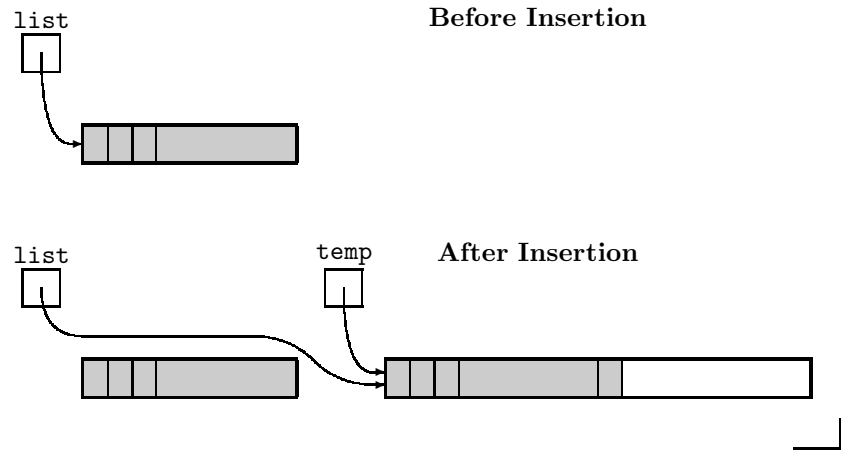
Example 2

The fragment shown here adds an item to a partially filled array called `list`. If there is no room in `list`, the fragment doubles its size before adding the new element.

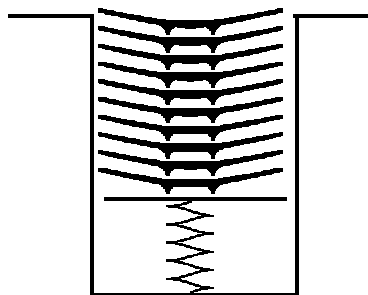
```
if (listSize == list.length)
{
    // array too small - double its size
    double[] temp = new double [2*list.length];
    for (int i = 0; i < list.length; i++)
        temp[i] = list[i];
    list = temp;
}
```

```
list[listSize] = item;
listSize++;
```

The diagrams show the effect of adding an element if the array is already fully occupied. (Occupied cells are indicated by the shaded regions.)



A *stack* is a list whose size can grow and shrink by inserting or deleting items at one end of the list. The end at which insertions and deletions are made is called the *top* of the stack; the other end is called the *bottom* of the stack. Stacks have a physical analogy in the spring-operated mechanisms sometimes used to store stacks of plates in cafeterias. In these devices, if you place a new plate on the top of the stack, the added pressure pushes the stack down a bit; if you remove a plate from the top, the decreased pressure allows the remaining plates to pop up a bit.



Following this analogy, to insert an item into a stack, we *push* it onto the stack; to delete an item, we *pop* it from the stack.

To implement stacks in Java, we can use partially filled arrays.⁴ For our stacks, we want to be able to perform the following operations:

- `create` create a new, empty stack
- `push` add a new item onto the top of the stack
- `pop` delete and return the item at the top of the stack
- `isEmpty` return true if and only if the stack is empty

In the class `Stack` that follows, when we create a new stack, we start it with an array that will hold ten items. If the size ever proves to be inadequate, we double the stack's capacity. Notice that the array that is used to hold the elements of the stack and its size field are both made `private`. This ensures that a user cannot manipulate the array directly. If we ever decide to change the implementation, it will not affect the way that the user interacts with the class.

Example 3

The following class implements the concept of a stack using partially filled arrays of type `Object`. An attempt to delete an item from an empty stack (in the `pop` method) will cause an exception to be thrown. The exception will print a message and execution of the program will terminate. For information on exceptions, see Appendix E.

```
class Stack
{
    private Object[] stack;
    private int stackSize;

    public Stack ()
    {
        // create a new, empty stack
        stack = new Object[10];
        stackSize = 0;
    }
}
```

⁴In Chapter 12, we will examine a very different way of implementing stacks.

```

public void push (Object o)
{
    // insert object o into stack
    if (stackSize == stack.length)
    {
        // array too small - double its size
        Object[] temp = new Object [2*stack.length];
        for (int i = 0; i < stack.length; i++)
            temp[i] = stack[i];
        stack = temp;
    }
    stack[stackSize] = o;
    stackSize++;
}

public Object pop ()
{
    // remove and return object at top of stack
    if (stackSize == 0)
        throw new RuntimeException
            ("Attempt to pop empty stack");
    else
    {
        stackSize--;
        return stack[stackSize];
    }
}

public boolean isEmpty ()
{
    // return true if and only if the stack is empty
    return stackSize == 0;
}
}

```

Java actually supplies a `Stack` class in the package `java.util` with methods for each of these operations plus some others. The implementation used by Java is similar to that used here.

Exercises 8.5

1. Complete the definition of the method `get` so that it returns the k^{th} element from a partially filled array of objects. If the array does not have a k^{th} element, the method should return `null`.

```
public static Object get (Object[] list,  
                          int listSize, int k)
```

2. Write a fragment that prints the largest value in a partially filled array of `double` values called `scores`. If the array is empty, the fragment should print an appropriate message. Assume that the `int` variable `scoresSize` contains the current size of the array.
3. Write a complete program that repeatedly prompts a user for positive integers and reads those values until the user supplies a value of zero. The program should then print the values read, in the order that they were supplied by the user. The program should not place a restriction on the number of values that can be read.
4. Another possible operation on stacks is usually called `peek`. It notes and returns the value of the item currently at the top of the stack. Write a method `peek` that could be added to the class `Stack` shown in Example 3. The method should have header

```
public Object peek ()
```

If the stack is not empty, the method should return a reference to the object at the top of the stack. Otherwise it should throw an exception. The method should not alter the contents of the stack.

8.6 Avoiding Errors and Debugging

Avoiding Errors

1. By far the most common error in using arrays is allowing an index to wander out of its defined range. Although this happens to everybody sooner or later, you can reduce the chance of it happening to you by (almost) always using `for` statements to process arrays. In setting

up such statements, use the array's length to control the loop. For example, if processing the elements of an array called `list`, write

```
for (int i = 0; i < list.length; i++) ...
```

2. Even if an operation on an array has some conditional aspect, it is still usually best to use a `for` statement rather than a `while` or a `do`. As an example, suppose we have an array called `list` and we want to set the variable `negativeIndex` to the index of the first negative value in `list` (or `-1` if there are no negative values in the array), we could write

```
int negativeIndex = -1;
boolean noNegative = true;
for (int i = 0; i < list.length && noNegative; i++)
    if (list[i] < 0)
    {
        negativeIndex = i;
        noNegative = false;
    }
```

3. One sure way to avoid errors with arrays is not to use them. Although this is often an unrealistic choice, there are many problems for which a solution without arrays is both possible and simpler. Before automatically creating an array, ask yourself if it is really necessary to do so. For example, if we want to read a sequence of values and determine the largest, it is not necessary to use an array because we never need all the values at once.

Debugging

1. In debugging a program that uses arrays, it is sometimes a good idea to (temporarily) reduce the size of the arrays to very small values until the error can be found and eliminated.
2. To trace the activity in an array, print both the components of the array and the associated index values. For example, to see what is occurring in an array called `list`, we could write

```
for (int i = 0; i < list.length; i++)
    System.out.println("list[" + i + "] = " + list[i]);
```

In this way, you can see not only what the values are but where they are located.

3. As with all objects, there are two parts to the creation of an array: declaration of a reference to the array and allocation of storage for the array itself. For example, the sequence

```
int [] list;
list = new int[10];
```

first creates `list`, a reference to an `int` array and then creates an array of `int` values (all initialized to 0) with the variable `list` acting as a reference to the array. The two operations can be (and usually are) combined into one statement

```
int [] list = new int[10];
```

but you should be clear about the two operations that are being performed by this single statement. If you forget to allocate space for the array, the compiler will catch you. For example, the fragment

```
double [] list;
list[0] = 1;
```

will cause the compiler to produce the message:

```
variable list may not have been initialized
```

4. This problem of forgetting to initialize arrays is very common with arrays of objects. As an example, the fragment

```
Object[] list = new Object[10];
System.out.println(list[0].equals(list[1]));
```

will compile without warning but will cause Java to throw a `NullPointerException` when it attempts execution. The problem is that `new` here creates an array of null references. An attempt to use the instance method `equals` when `list[0]` is not referring to an instance of an object naturally causes Java to get upset.

Exercises 8.6

1. The following fragment determines the largest number in `list`, an array of `double` values. Rewrite the fragment to make it clearer.

```
double largest = list[0];
int i = 1;
while (i < list.length)
    if (list[i++] > largest)
        largest = list[i-1];
```

2. Suppose that you are required to read a set of values and determine the given quantity. For which ones (if any) would you need to use an array?

- (a) the largest value
- (b) the median
- (c) the mean
- (d) the range

3. Identify and correct the error in each declaration.

- (a) `double a = new double[10];`
- (b) `int[] b = new int[];`
- (c) `char[] [] c = char[30] [];`
- (d) `float[] [] d = new float[] [10];`
- (e) `int[] e = new {3,5,2,9,1};`

4. A programmer, using a square two-dimensional array of `int` values called `table`, wanted to sum the elements along the *main diagonal* (the diagonal whose elements are `table[0][0]`, `table[1][1]`, and so on). To do this the programmer wrote

```
int total = 0;
for (int i = 0; i < list.length; i++)
    for (int j = 0; j < list.length; j++)
        total += table[i][j];
```

- (a) What does the fragment actually do?
- (b) Write a fragment that does set `total` to the sum of the elements of the main diagonal.
- (c) Write a fragment that sets `total` to the sum of the elements of the other diagonal of `table`.

8.7 Review Exercises 8

1. Does the statement `double[] list;` create an array of `double` elements? Explain.

2. State the value of the indicated element after execution of the given declaration.
 - (a) `a[4]` after `int[] a = new int[20];`
 - (b) `b[23]` after `boolean[] b = new boolean[50];`
 - (c) `c[2]` after `int[] c = {4,7,2,8};`
 - (d) `d[33]` after `double[][] d = new double[100][];`
3. How much space (in bytes) would be required to store the elements of the arrays created by the following declarations?
 - (a) `double[] a = new double[10];`
 - (b) `byte[] b = new byte[30];`
 - (c) `int[][] c = new int[10][20];`
 - (d) `char[][][] d = new char[5][4][50];`
4. Given that `list` is a one-dimensional array of `int` values, write fragments to print each value.
 - (a) the number of occurrences of the value zero,
 - (b) the product of all the elements,
 - (c) the sum of the positive elements,
 - (d) the minimum value.
5. Given that `table` is a two-dimensional rectangular array, write fragments to print each value.
 - (a) the number of elements
 - (b) the sum of the elements,
 - (c) the number of values that are multiples of three,
 - (d) the positive difference between the largest and smallest values in the array.
6. A *polynomial* in x of *degree* n is an expression of the form

$$a_0 + a_1x + a_2x^2 + \cdots + a_nx^n \quad (a_n \neq 0)$$

The values $a_0, a_1, \dots, a_n$ are called the *coefficients* of the polynomial. Complete the definition of the method `eval` so that it returns the value at `x` of a polynomial whose coefficients are stored in the array `a`.

```
public static double eval (double[] a, double x)
```

7. Complete the definition of the method `randomize` whose header is
- ```
public static int[] randomize (int n)
```
- The method should return an array of size `n` whose elements are the values  $0 \dots n - 1$  ordered randomly. As an example, `randomize(5)` might return `[4, 2, 0, 3, 1]`.
8. *Lacsap's Triangle* is a triangular array of numbers in which each row starts and ends with the row number and each interior value is the sum of the two values on either side of it in the preceding row. The following diagram shows the first five rows of Lacsap's Triangle.

```

 1
 2 2
 3 4 3
4 7 7 4
5 11 14 11 5

```

Write a program that asks the user for a positive integer and, once a satisfactory value has been supplied, produces that many rows of Lacsap's Triangle. For simplicity, do not try to print the triangle in the symmetrical form shown here.

9. A *matrix* is a rectangular array of values. The *transpose* of a matrix is the matrix obtained by interchanging the rows and columns of the original matrix. As an example,

$$\text{the transpose of } \begin{bmatrix} 3 & 1 & 4 \\ 2 & 0 & 7 \end{bmatrix} \text{ is } \begin{bmatrix} 3 & 2 \\ 1 & 0 \\ 4 & 7 \end{bmatrix}$$

Write a Java method called `transpose` that has one parameter, a two-dimensional array of `int` values. The method should return a two-dimensional array that is the transpose of the original array. Assume that the parameter is a rectangular array.

10. A *magic square* of order  $n$  is a square array containing the integers  $1, 2, \dots, n^2$ , each one used exactly once. The values in each row, column, and diagonal must sum to the same value. As an example, the following array is a magic square of order 3.

```

4 3 8
9 5 1
2 7 6

```

Write a `boolean` method `isMagic` that returns `true` if and only if its single `int[] []` parameter represents a magic square.

## Projects

11. The *Sieve of Eratosthenes* is the name given to an algorithm<sup>5</sup> for finding *prime numbers* (natural numbers having exactly two distinct natural number divisors, one and themselves). The sieve starts by considering all numbers greater than one as possible primes.

2 3 4 5 6 7 8 9 10 11 12 13 14 15 ...

The first number in this list, 2, must be prime but all multiples of two cannot be prime and hence can be eliminated from the list to give us

2 3 ~~4~~ 5 ~~6~~ 7 ~~8~~ 9 ~~10~~ 11 ~~12~~ 13 ~~14~~ 15 ...

The next number still in the list, 3, must also be prime since it was not eliminated when we found multiples of two. We can, however, eliminate all multiples of three to obtain

2 3 ~~4~~ 5 ~~6~~ 7 ~~8~~ ~~9~~ ~~10~~ 11 ~~12~~ 13 ~~14~~ ~~15~~ ...

In the algorithm, this process continues (eliminating multiples of five, seven, and so on) until it is no longer possible to eliminate any multiples of values left in the table. The values that remain must all be primes.

Write a program to implement Eratosthene's Sieve. The program should ask the user for an upper bound and should then print all the prime numbers less than or equal to that upper bound. In your program, use a `boolean` array to maintain a record of the numbers that are possible primes.

The output should have eight values per line with each value right-justified in a field that is ten characters wide. You may find the methods of the `Out` class shown in Appendix A useful for this.

---

<sup>5</sup>An *algorithm* is a set of instructions for performing some process in a finite number of steps.

12. (a) The *mode* of a group of values is the value that occurs most often in the group. A group may have more than one mode if more than one value occurs with the maximum frequency. Such a group is said to be *multi-modal*. (If there are exactly two modes, the values are said to be *bi-modal*.) Write a program that will read an unknown number of marks (out of 100) quitting reading when a sentinel of  $-1$  is read. The program should then determine and print the mode(s) of the marks.
- (b) Modify the program so that it also computes the *median*. If there is an odd number of ordered values, the median is the middle one; if there is an even number of values, the median is the mean of the two scores adjacent to the middle. For example, for scores of 51 68 68 73 84, the median is 68 while for scores of 51 68 68 73 84 90, the median is 70.5 (the mean of 68 and 73).
13. Write a program that first reads the positions of a number of pieces on a chess board and then reads the location of another position. A piece can attack a position if it can move into that position. Your program should determine which of the pieces can attack the given position.

Chess is played on an  $8 \times 8$  board whose rows are numbered from 1 to 8 (from the bottom to the top) and whose columns are lettered from a to h (from left to right). In this problem you are only going to be dealing with the following pieces:

- King (K): can move one space in any direction
- Queen (Q): can move any number of spaces in any direction
- Rook (R): can move any number of spaces, either horizontally or vertically
- Bishop (B): can move any number of spaces diagonally
- Knight (N): can move two spaces vertically followed by one space horizontally or two spaces horizontally followed by one space vertically

The program should read the position of a piece by reading the piece code, the column, and the row in that order, one character at a time. It should continue to read the positions of pieces until the piece code is X, indicating the position to be attacked. Assume that all input is valid.

The output from the program should consist of a visual representation of the chess board, as shown in the example. The diagram should show the pieces in their positions, the position under attack (denoted by the letter **X**) and dots in empty positions. The diagram should be followed by a list of the pieces that can take a piece at the location marked by the **X**. A piece is indicated by its code, its column, and its row, in that order. The attacking pieces should be listed in the following order: from the bottom row to the top row and from left to right within a row. To determine whether or not a piece is able to attack a position, ignore any other pieces that may be in the way.

Sample input for the program is shown here. (Although all input is shown on one line, when the program runs, each character should be on one line.)

Q b 2 R h 4 K e 4 B c 4 X d 4

For such input, the program should produce the following output.

```

.....
.....
.....
.....
..BXK..R
.....
.Q.....
.....
Qb2
Ke4
Rh4

```

14. Write a program that permits two players to play the game of *Adjacency*. The game is played using **X**'s and **O**'s on an  $8 \times 8$  board, initially set up as shown in the following diagram in which dots indicate empty squares. The rows and columns are numbered as shown.

```

800
700
6
5
4
3
2 XX.....
1 XX.....
12345678

```

The first player begins by placing an **X** on an empty square. All **O**'s that are on adjacent squares are then replaced with **X**'s. Two squares are considered to be adjacent if they are one square apart either horizontally, vertically, or diagonally. To illustrate, if the first player puts an **X** on the square with coordinates (6, 7), column 6 and row 7, the resulting board configuration would be:

```

8XO
7XXO
6
5
4
3
2 XX.....
1 XX.....
12345678

```

Play continues with the second player placing an **O** on an empty square. The two players then take alternate turns. The game can be terminated in one of two ways. The players may agree before the first turn to terminate the game after a specific number of rounds or play may continue until the board is filled. At the end of the game, the player with the most pieces is the winner.

Your program should begin by asking for the names of the players, determining the way in which they want the game to terminate, and displaying the initial board. It should then repeatedly prompt the players for their moves, displaying the updated board after each move. If a player supplies invalid coordinates for a move, the program should spot this and ask the player to re-submit valid coordinates immediately. At the end of the game, the program should announce either the name of the winner, if there is one, or print a message in case of a tie.

## Real Jobs — Real People

**Name:** Geoffrey Cann  
**Title:** Partner  
**Company:** Deloitte Consulting  
**Education:** B.Comm. Management Information Systems,  
McGill University (1984)  
M.B.A., University of Western Ontario (1989)

Q: Please describe a bit about your career history.

A: While I was an undergraduate, I worked as a subcontractor to small software development companies building applications for a new IBM technology called the “Personal Computer”. After that, I worked for Imperial Oil as a programmer building financial and marketing systems in COBOL and other mainframe languages.

After completing my MBA, I joined Deloitte Consulting where I was initially involved in writing technical specifications for word processing, procurement, email, and other systems. Then I got into developing information technology (IT) strategies and helping organizations build custom-made systems for core functions.

In 1993, I relocated to Hong Kong to work with a small Deloitte Consulting team to develop a consulting practice in the region. I did a lot of systems work for clients in Hong Kong and mainland China. In 1995, I moved to Western Canada and focused on evaluation and selection of large enterprise resource planning systems like SAP and PeopleSoft.

Since 1999, I have been working primarily with a large Canadian energy company. It is going through a major transformation and I am helping with all aspects of its business plans and information systems.

Q: What are some of the key IT challenges facing organizations you work with?

A: One of the biggest challenges for many organizations is the rapid pace of change. Technology changes too quickly for most companies to be able to embrace and implement the latest advancements. Since it is so difficult to keep up, companies are often one generation behind both from a systems and a business model perspective.

Another major challenge organizations face is dependence on an aging group of programmers who built many of their core systems. As these individuals retire they often leave huge gaps in knowledge.

A third challenge facing many companies is the emergence of third world economies with outstanding software development capabilities. Com-

panies need to figure out how to take advantage of these resources and integrate them effectively into their business models. For example, many organizations are now outsourcing their customer support to call centres in India – this has enormous potential but also creates new challenges around customer service.

Q: How do you help an organization look at its IT Strategy? What are the key elements you focus on?

A: First and foremost, you have to look at the business strategy and ensure the IT strategy supports the prime goals and objectives of the company. Second, I like to get an external point of view, looking for examples from other industries of organizations that have made excellent use of a particular technology or approach. Third, I recommend that organizations tie their IT strategy development cycle to the business planning cycle.

I like to look at IT systems from a portfolio perspective. Each project needs to be constantly monitored and managed to ensure that it is still supporting the business plans. Organizations often fall into the trap of continuing to invest in projects that no longer meet core business requirements.

Q: What do you think about Java?

A: It is a very powerful programming language and the portability to multiple devices (*i.e.* “write once, use many”) is extremely valuable. I think that the real power will come when organizations really embrace wireless applications.

Q: What career advice do you have for someone studying computing science today?

A: First, take advantage of opportunities to obtain business knowledge; the better you understand business issues, the more effective you will be in any IT career. Second, I recommend not getting too focused on a single technology because it will change. Third, try to get as much diverse experience as possible, perhaps working for a big company for a while and then going to a small software shop; you will learn valuable lessons from both kinds of environments.